PYTHON NOTES

DAY 10
Loops & Iteration

Clear concepts • Short examples • Quick revision

Learning goal: understand loop control, nested loops, range(), enumerate(), zip(), and the basics of iterators.

Contents

01	pass statement and loop else
02	Nested loops and range()
03	enumerate() and zip()
04	Iterators, next(), and StopIteration
05	Quick revision

01 · Loop Control

The pass statement

pass is a placeholder. It does nothing, but it keeps a code block valid while you plan to add code later. Unlike **break** and **continue**, it does not change the flow of a loop.

```
for number in range(1, 4):
    if number == 2:
        pass
    print(number)
```

Output:
```
1
2
3
```

Compare the statements

Statement	Effect	Typical use
break	Ends the loop immediately	Stop after finding a value
continue	Skips to the next iteration	Skip a value or condition
pass	Does nothing	Placeholder for future code

The else block with loops

A loop's else block runs when the loop finishes normally. If the loop is stopped by **break**, the else block is skipped.

```
for number in range(1, 4):
    print(number)
else:
    print("Loop finished")
```

Output:
```
1
2
3
Loop finished
```

```
for number in range(1, 5):
    if number == 3:
        break
    print(number)
else:
    print("Loop finished")
```

Output:
```
1
2
```

Remember: loop else is linked to normal completion, not to an if/else condition.

02 · Nested Loops & range()

Nested loops

A nested loop is a loop inside another loop. For every one iteration of the outer loop, the inner loop completes its own iterations. Nested loops are useful for tables, rows, columns, and patterns.

```
for row in range(1, 3):
    for column in range(1, 4):
        print(row, column)
```

Output:
```
(1, 1), (1, 2), (1, 3)
(2, 1), (2, 2), (2, 3)
```

The range() function

range() generates a sequence of integers. The stop value is not included.

Form	Example	Values
range(stop)	range(4)	0, 1, 2, 3
range(start, stop)	range(2, 5)	2, 3, 4
range(start, stop, step)	range(1, 8, 2)	1, 3, 5, 7
Negative step	range(5, 0, -1)	5, 4, 3, 2, 1

```
for number in range(5, 0, -1):
    print(number)
```

Output:
```
5
4
3
2
1
```

Tip: choose a positive step to count upward and a negative step to count downward.

03 · enumerate() & zip()

enumerate()

enumerate() gives each item together with its index. By default, counting starts at 0; you can provide a different start value.

```
names = ["Asha", "Ravi", "Neha"]

for index, name in enumerate(names, start=1):
    print(index, name)
```

Output:

```
1 Asha
2 Ravi
3 Neha
```

zip()

zip() combines items from two or more iterables position by position. By default, it stops when the shortest iterable runs out of items.

```
names = ["Asha", "Ravi", "Neha"]
scores = [80, 92, 87]

for name, score in zip(names, scores):
    print(name, score)
```

Output:

```
Asha 80
Ravi 92
Neha 87
```

Use enumerate() when you need an index. Use zip() when you want to process related lists together.

04 · Iterator Basics

Iterable vs iterator

Term	Meaning	Example
Iterable	An object you can loop over	list, string, tuple
Iterator	An object that returns the next item when requested	An iterator made with iter()

iter() and next()

iter() creates an iterator from an iterable. next() retrieves the next available item from that iterator.

```
numbers = [10, 20, 30]
items = iter(numbers)

print(next(items))
print(next(items))
print(next(items))
```

Output:
```
10
20
30
```

StopIteration

When an iterator has no items left, calling next() again raises StopIteration. A for loop handles this condition automatically.

```
numbers = [10, 20]
items = iter(numbers)

print(next(items))
print(next(items))
# next(items) again raises StopIteration
```

Output:
```
10
20
```

Infinite loops

An infinite loop keeps running because its stopping condition never becomes false. Make sure the loop has a suitable condition or a deliberate exit.

```
count = 1
while count <= 3:
    print(count)
    count += 1
```

Output:
```
1
2
3
```

05 · Quick Revision

Topic	Remember this
pass	Placeholder; no effect on program flow.
break	Immediately exits the nearest loop.
continue	Skips to the next loop iteration.
Loop else	Runs if the loop completes without break.
Nested loop	An inner loop runs inside an outer loop.
range()	Creates a sequence; stop is excluded.
enumerate()	Provides index and item.
zip()	Pairs items from multiple iterables.
iter()	Creates an iterator from an iterable.
next()	Returns the next item from an iterator.
StopIteration	Signals that no iterator items remain.

Before finishing: practise writing each example without looking at the solution, then explain the output in your own words.